TU WIEN Informatics

Towards Interoperability between Agentic AI Frameworks through Semantic Representation

DIPLOMA THESIS

submitted in partial fulfillment of the requirements for the degree of

Diplom-Ingenieur

in

Business Informatics

by

Dani Lippmann BSc
Registration Number 12332231

to the Faculty of Informatics

at the TU Wien

Advisor: Forename Surname
Assistance:

Vienna, June 30, 2026

Dani Lippmann BSc	Forename Surname

Declaration of Authorship

Dani Lippmann BSc

I hereby declare that I have written this thesis independently, that I have completely specified the utilized sources and resources and that I have definitely marked all parts of the work – including tables, maps and figures – which belong to other works or to the internet, literally or extracted, by referencing the source as borrowed.
I further declare that I have used generative AI tools only as an aid, and that my own intellectual and creative efforts predominate in this work. In the appendix "Overview of Generative AI Tools Used" I have listed all generative AI tools that were used in the creation of this work, and indicated where in the work they were used. If whole passages of text were used without substantial changes, I have indicated the input (prompts) I formulated and the IT application used with its product name and version number/date.

Vienna, June 30, 2026

Dani Lippmann BSc

Acknowledgements

Abstract

Contents

CHAPTER 1

Introduction

CHAPTER 2

Related Work

2.1 Agentic AI Systems

In contrast to standard AI systems, agentic systems are defined as

2.2 Agentic AI Frameworks

2.2.1 LangGraph

2.2.2 CrewAI

2.2.3 AutoGen

2.3 Semantic Representations for AI Systems

2.3.1 AgentO

2.4 Research Gap

CHAPTER 3

Research Methodology

CHAPTER 4

Agentic AI Concepts and Framework-level Implementation

Researchers have studied Multi-Agent Systems (MAS) and their architectures for decades. [Woo09] established the groundwork for the field by defining core concepts like reasoning, agent interactions, coordination, and communication. Although this work was written before the current rise of generative AI, it still provides a solid foundation for understanding how modern agentic systems are designed. Recent studies have updated these classical ideas to fit LLM-based agents. This includes adding new elements like tool use, memory architectures, and how tasks are delegated [DBM25, LWZ+24, CLH+25, WZ25, Sch25, Bot25].

[DBM25] created a unified model that shows the main parts of an agentic system (see Figure 4.1). Other researchers, such as [Bot25], suggest that the core of these systems can be simplified into three main pillars: Reasoning, Memory, and Tool Use. Currently, there is no single, agreed-upon way to categorize these designs in the research community. Because of this, this paper proposes its own taxonomy 4.1. By combining classic MAS principles with modern AI research, this taxonomy provides a clear starting point for the framework level analysis.

These concepts are then further analysed at the source code level across three agentic AI frameworks representing three distinct architectural paradigms: sequential role-based (CrewAI), graph-based (LangGraph) and conversation-driven (AutoGen). These three frameworks were selected because they represent fundamentally different approaches to structuring agent interactions and control flow, providing the necessary heterogeneity to derive framework-agnostic requirements [Cre25] [Mic25] [Lan25]. These requirements directly guide the subsequent ontology engineering phase of this research.

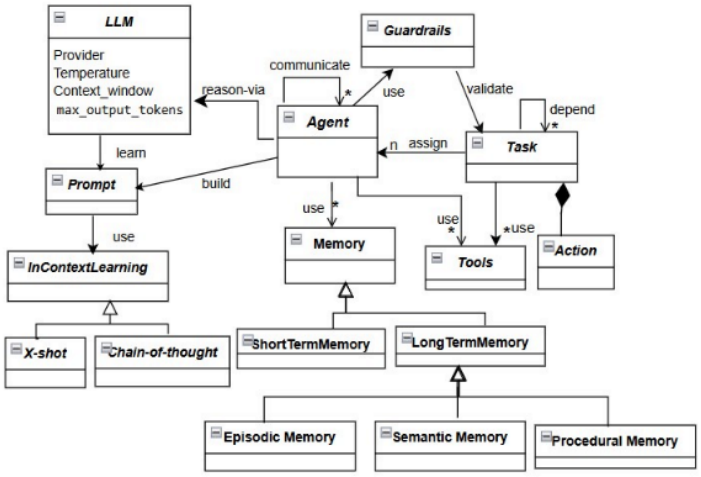

Figure 4.1: Unified class model showing the core components of agentic AI systems [DBM25]

Table 4.1: Concept Matrix — Concepts for Agentic AI Frameworks

	Agent Definition			Reasoning Engine			Memory Architecture			Perception Layer		Tool Integration		Coordination		Communication		Guardrails			
Paper	Agent Profile Schema	Agent Role Config.	LLM Backbone Sel.	Planning & Decomp.	Reflection Mechanism	Learning Strategy	Knowledge Store	Short-Term Memory	Long-Term Mem./RAG	Env. Interface Design	Input Processing	Tool Reg. & Func. Schema	Action Space Def.	Orchestration & Routing	Multi-Agent Topology	Agent-to-Agent Msg.	HITL Interface	Rule & Constraint Def.	Intervention Mech.	Prompt Template	**Count**
Schneider (2025)	•	•		•	•		•	•		•	•			•	•		•	•			12
Guo (2024)	•									•						•					3
Horling & Lesser (2005)															•						1
Botti (2025)				•			•					•									3
Li et al. (2024)	•			•			•	•		•		•									6
Microsoft	•						•		•			•		•							5
Derouiche et al. (2025)	•		•			•	•	•				•		•					•	•	9
Chen et al. (2025)	•		•	•			•	•		•		•					•	•	•		10
Wissuchek et al. (2025)		•		•	•		•		•	•	•	•	•								9
Deepak et al. (2025)				•		•	•	•		•		•									6
Wooldridge (2002)	•			•											•	•					4
Wooldridge & Jennings (1995)	•																				1
Nisa et al. (2026)				•	•							•			•						4
Yehudai et al. (2025)				•	•		•					•									4
Bandi et al. (2025)				•	•		•			•	•		•	•		•					8
Total	**8**	**2**	**2**	**10**	**5**	**2**	**10**	**5**	**3**	**6**	**3**	**9**	**2**	**4**	**4**	**3**	**2**	**2**	**2**	**1**	**85**

4.1 Agent Definition

[WJ95] provide one of the earliest comprehensive surveys of agent theories, architectures and languages, establishing core properties such as autonomy, reactivity, proactivity and social ability as defining characteristics of an agent. They define agents as a computing system situated in an environment that is capable of flexible, autonomous action to meet its design objectives.

In the context of modern LLM-based multi-agent systems, the agent remains a central architectural component, but its characterization has changed from simple reactive/proactive agents to fully self-aware agents, that exhibit meta-cognitve awareness [NSSP26].

In the context of the modern AI landscape, an agent is based on a foundation model (like a Large Language Model) and pursues complex, multi-step goals with limited direct supervision [Sch25]. Furthermore, these agents are often defined with specific profiles, roles, backgrounds and constraints that specify how they act within the agentic AI system [LWZ+24] [GCW+24].

4.1.1 Framework Analysis

CrewAI

CrewAI defines an Agent as an autonomous unit capable of performing tasks, making decisions, using tools, interacting with other agents, maintaining memory of interactions and being able to delegate tasks when allowed. In the source code, agents are defined either using YAML configuration or directly in code.To define an agent's profile, CrewAI provides a list of agent attributes that can be specified when designing an agentic system. The only required parameters are role, goal and backstory [Cre25].

```yaml
data_analyst:
  role: Data Analyst
  goal: >
    Extract actionable insights from structured datasets
    and surface relevant patterns to the research team.
  backstory: >
    A seasoned analyst with expertise in statistical methods
    and business intelligence who prioritises clarity over
    complexity when communicating findings.
```

Listing 4.1: CrewAI agent definition (YAML configuration).

AutoGen

AutoGen provides a set of preset agent types that all inherit from the BaseChatAgent class. All agents share a common set of attributes that define their identity: name, description and system_message, which are configured at agent instantiation. The name serves as a unique identifier within a multi-agent system, the description communicates the agent's purpose to other agents during orchestration and the system_message provides the behavioral instructions passed to the underlying language model.

Among the preset agents, AssistantAgent serves as a general-purpose agent intended for prototyping and educational use. For more specialized requirements, AutoGen allows developers to define custom agents by subclassing BaseChatAgent, enabling the implementation of domain-specific behavior that does not fit the provided presets [Mic25].

LangGraph

LangGraph, while part of the LangChain ecosystem, follows a fundamentally different paradigm at the source code level than LangChain's agent abstractions. LangGraph

explicitly distinguishes between *workflows* and *agents*: workflows follow predetermined code paths with a fixed execution order, whereas agents are driven by a language model that dynamically determines its own sequence of actions, typically through iterative tool use.

LangGraph does not provide its users with a mechanism to configure specific agents with roles, descriptions, or backstories. Instead, behavior is constructed using discrete steps called *nodes*, connected via *edges* that define transitions and conditional branching. An agent in LangGraph is therefore not defined through identity parameters but rather emerges as a node in which a language model autonomously decides whether to invoke tools or return a final response, typically implemented through a react-style loop [Lan25].

```
1
2 def data_analyst(state: MessagesState):
3     system = SystemMessage(content=(
4         "You are a data analyst with expertise in statistical "
5         "methods and business intelligence. Prioritise clarity "
6         "over complexity when communicating findings."
7     ))
8     response = model.invoke([system] + state["messages"])
9     return {"messages": [response]}
```

Listing 4.2: LangGraph agent definition as a node function

4.1.2 Synthesis and Requirements

The framework analysis of agent definitions revealed structural inconsistencies on an architectural level across all three frameworks. While CrewAI and AutoGen support defining individual agents with distinct characteristics and roles through dedicated parameters, LangGraph follows a fundamentally different paradigm in which agents are not explicitly declared but emerge from graph nodes that contain language model calls with tool access.

Despite these differences, all three frameworks share the underlying concept of an autonomous unit that is associated with a language model and guided by natural language instructions. CrewAI expresses this through role, goal and backstory; AutoGen through name, description and system_message and LangGraph implicitly through prompt content within node functions.

From this analysis, the following representational requirements for agent definitions can be derived:

R1 Framework-independent agent identity. The ontology must represent agent identity as a separable concept that can be populated either from explicit configuration attributes (as in CrewAI and AutoGen) or inferred from prompt-level content within computational nodes (as in LangGraph).

R2 Distinction between orchestration-relevant and model-relevant attributes. The ontology must distinguish between agent attributes that serve the orchestration layer (e.g., AutoGen's description used for speaker selection) and those that instruct the language model (e.g., AutoGen's system_message, CrewAI's backstory). A single agent may carry both types of directive simultaneously.

R3 Prompt decomposition into structured components The ontology must decompose natural language prompt content into structured components so that equivalent prompts can be reconstructed for different target frameworks from these components.

R4 Flexible attribute cardinality. The required and optional attributes differ across frameworks. CrewAI mandates role, goal and backstory; AutoGen requires only name; LangGraph has no required agent-level attributes at all. The ontology must define a minimal set of identity properties while accommodating framework-specific extensions without loss of information.

R5 Agent functional typing. The ontology must represent the functional role an agent plays within the coordination structure by distinguishing between general-purpose agents, manager/orchestrator agents, and agents that serve as proxies for human input (independently of how this role is expressed in framework-specific class hierarchies).

4.2 Perception Layer

The perception layer is responsible for ingesting external inputs from the environment and translating them into internal representations that the agent can reason about [BKN+25]. While early foundation models relied almost exclusively on text, modern agentic systems are increasingly equipped with multi-modal perception capabilities, processing inputs including textual descriptions, visual images, auditory signals and spatial coordinates [LWZ+24] [WZ25].

Critically, agentic systems do not rely on rigid, explicit commands; instead, they act upon abstract, high-level user goals. The perception layer translates these goals, alongside multi-modal context, into structured internal representations that drive the agent's behavior [BKN+25]. It therefore constitutes the boundary at which human intent enters the agent system and takes on a computationally tractable form.

In the broader agentic AI literature, the perception layer describes how agents ingest and internalize external input at runtime. At the design-time abstraction level addressed by this thesis, the concern is how developers specify the tasks and actions an agent system should perform [DBM25]. The following analysis examines how the three frameworks define tasks, assign them to agents and express dependencies and output expectations.

4.2.1 Framework Analysis

CrewAI

CrewAI lets developers define a Task as a specific assignment completed by an agent. Similar to the Agent , tasks can be defined as YAML configuration or directly in the code.

On a source code level, a task is created with two required parameters: description, a natural language statement of what the task entails and expected_output, a description of what the completed task should produce. Tasks are optionally assigned to a specific agent via the agent parameter; if no agent is assigned, a hierarchical process can delegate tasks based on agent roles.

Tasks can depend on other tasks through the context attribute, which accepts a list of tasks whose outputs should be available as input. CrewAI also supports task-level tool assignment, allowing a task to override the agent's default tools, as well as output formatting through structured types such as output_pydantic and output_json [Cre25].

AutoGen

In AutoGen, there is no dedicated task class or configuration object. Instead, a task is expressed as a message passed to an agent or team at through the run or run_stream methods via the task parameter, which accepts a plain string. The task description, expected output and any constraints must all be encoded within this single string. There is no built-in mechanism for defining task dependencies, structured output formats, or guardrails at the task level; such logic must be implemented through agent orchestration or custom code [Mic25].

LangGraph

LangGraph does not have an explicit task concept. What other frameworks model as a task is instead encoded in the graph structure itself: a node's function body contains the instructions (typically as part of the prompt passed to the language model) and the graph's edges define the execution order and dependencies between steps. Task descriptions, expected outputs and constraints are embedded within node implementations rather than declared as separate, configurable objects [Lan25].

4.2.2 Synthesis and Requirements

The framework analysis of task definitions revealed three distinct abstraction levels. CrewAI provides tasks as first-class, richly configurable objects with dedicated attributes for descriptions, expected outputs, agent assignment, dependencies, guardrails and structured output types. AutoGen reduces a task to a plain string passed at runtime, with no structural separation between the task description and its constraints. LangGraph has no explicit task concept; task logic is distributed across node functions and graph edges.

From this analysis, the following representational requirements can be derived:

R6 Task as a first-class ontological concept. The ontology must represent tasks as distinct entities with their own properties, even when the source framework does not model them explicitly. For AutoGen, this requires extracting task semantics from run method; for LangGraph, from node function bodies and associated prompts.

R7 Task-agent assignment. The ontology must represent the relationship between a task and the agent responsible for its execution. This includes explicit assignment (CrewAI's agent parameter), implicit assignment through orchestration (AutoGen's team routing) and structural assignment through graph topology (LangGraph's node-to-function binding).

R8 Task dependencies. The ontology must capture ordering and data dependencies between tasks. CrewAI models these explicitly through the context attribute, LangGraph encodes them as graph edges and AutoGen relies on sequential team execution or manual orchestration. The semantic representation must normalize these into a unified dependency model.

R9 Task output. The ontology must represent the expected output of a task, both as a natural language description and, where available, as a structured schema definition.

4.3 Coordination Mechanism

When tasks exceed the capacity of a single agent, Agentic AI employs MAS, where specialized agents coordinate their efforts to solve complex problems [Bot25]. The architecture of this coordination typically falls into several structural categories: centralized orchestration, where a single supervisor agent decomposes tasks and routes them to sub-agents; hierarchical structures, where agents operate in a tree-like command flow; and decentralized or peer-to-peer networks, where agents negotiate and collaborate directly without a central authority [BKN+25] [LWZ+24].

Beyond structural organization, the mechanisms of coordination are defined by the interaction paradigms among agents, which can be cooperative, adversarial (debate), or mixed [LWZ+24]. In cooperative teamwork, agents share information and divide labor to reach a common goal. Conversely, adversarial debate involves agents presenting and defending conflicting viewpoints to cross-validate reasoning, reduce hallucinations and refine solutions [GCW+24]. To efficiently manage the flow of information during these collaborative processes, systems often utilize a Shared Message Pool, a publish-subscribe mechanism allowing agents to extract only the information relevant to their specific roles without requiring direct point-to-point communication [HZC+24].

4.3.1 Framework Analysis

CrewAI

CrewAI implements coordination at two distinct levels: *Crews* for organizing agents around a shared set of tasks and *Flows* for orchestrating multiple crews and coding tasks into larger workflows.

A Crew is a team of agents assigned to a list of tasks, governed by a *process* that determines execution order. CrewAI provides two process types: sequential, where tasks are executed in the order they are defined with the output of one task serving as context for the next and hierarchical, where a manager agent dynamically delegates tasks to other agents based on their roles and capabilities. The process type is set at crew creation via the process parameter. In hierarchical mode, either a manager_llm or a custom manager_agent must be specified.

Flows operate at a higher abstraction level, connecting multiple crews, individual agents and arbitrary Python functions into event-driven workflows. A Flow is defined as a Python class inheriting from Flow, with methods decorated as entry points (@start), listeners (@listen), or routers (@router). Listeners are triggered when a specified method completes, enabling sequential chaining. Conditional logic is supported through or_ (trigger when any dependency completes), and_ (trigger when all dependencies complete) and router methods that return string labels to direct execution to different branches. Flows maintain their own state, which can be either unstructured (dictionary) or structured (Pydantic model) and is accessible across all methods in the flow [Cre25].

AutoGen

AutoGen implements coordination through *teams*, which are groups of agents that share a conversation context and take turns responding according to a defined interaction pattern. A team is instantiated by passing a list of agents and a termination condition that determines when execution stops.

AutoGen provides several team presets, each implementing a different speaker selection strategy:

- RoundRobinGroupChat: Agents take turns in a fixed round-robin order. Each agent broadcasts its response to all other agents, maintaining a shared context.
- SelectorGroupChat: A language model selects the next speaker after each message, enabling dynamic, model-driven delegation.
- MagenticOneGroupChat: A generalist multi-agent configuration designed for solving open-ended web and file-based tasks.
- Swarm: Agents use HandoffMessage to explicitly signal transitions to specific other agents, enabling developer-defined handoff patterns.

```
1
2  termination = (
3      TextMentionTermination("DONE")
4      | MaxMessageTermination(max_messages=6)
5  )
6
7  team = RoundRobinGroupChat(
8      participants=[researcher, critic],
9      termination_condition=termination,
10 )
```

Listing 4.3: AutoGen `RoundRobinGroupChat` with combined termination conditions.

Termination conditions control when a team stops executing. These are passed at team creation and include conditions such as TextMentionTermination (stops when a specific word appears), TextMessageTermination (stops when a specific agent produces a text message) and ExternalTermination (stops from outside the team). Conditions can be combined using bitwise operators [Mic25].

LangGraph

LangGraph encodes all coordination logic explicitly through a directed graph composed of nodes and edges. There is no dedicated team, crew, or orchestration abstraction. The graph itself is the coordination mechanism. Nodes represent discrete computational steps (agent calls, tool execution, routing logic) and edges define the transitions between them. This gives developers full control over the execution flow but requires all coordination patterns to be manually constructed.

On a source code level, a graph is built using the StateGraph class, which is parameterized with a typed state schema. Nodes are added via add_node and transitions are defined through add_edge for unconditional transitions or add_conditional_edges for branching based on the output of a node. Conditional edges accept a routing function that inspects the current state and returns the name of the next node to execute.

LangGraph provides prebuilt utilities that implement common coordination patterns. The create_react_agent function constructs a ReAct-style loop where an agent node and a tool node alternate until the agent produces a final response. For multi-agent coordination, developers manually compose multiple agent nodes within a single graph and define the routing logic between them [Lan25].

4.3.2 Synthesis and Requirements

The framework analysis of coordination revealed three fundamentally different paradigms. Despite these differences, all three frameworks ultimately express the same fundamental coordination concerns: which agents participate, in what order they act and under what conditions execution branches or terminates. The difference is whether these concerns are

declared through configuration (CrewAI's process type), selected from presets (AutoGen's team types), or constructed from primitives (LangGraph's nodes and edges).

From this analysis, the following representational requirements can be derived:

R10 Coordination pattern as a named concept. The ontology must represent coordination patterns (sequential, hierarchical, round-robin, swarm, selector-based) as identifiable concepts, regardless of whether they are selected from a preset (CrewAI, AutoGen) or manually constructed in a graph (LangGraph). This enables cross-framework comparison at the architectural level.

R11 Agent-to-coordination binding. The ontology must capture which agents participate in a coordination structure and how they are bound to it. Either as members of a crew with assigned tasks (CrewAI), as participants in a team (AutoGen), or as nodes within a graph (LangGraph).

R12 Termination conditions. All three frameworks define conditions under which execution stops, though the mechanisms differ: CrewAI relies on task completion and flow structure, AutoGen uses explicit termination condition objects and LangGraph terminates when a node routes to the END sentinel. The ontology must capture termination semantics independently of these framework-specific mechanisms.

R13 Multi-level coordination. CrewAI's separation of crews (inner coordination) and flows (outer orchestration) has no direct equivalent in AutoGen or LangGraph, where a single abstraction handles both levels. The ontology must be able to represent nested coordination structures so that a CrewAI flow containing multiple crews can be captured without flattening the hierarchy, while also representing the single-level coordination of the other frameworks.

R14 Workflow step structure with typed transitions. The ontology must represent the execution structure of a coordination pattern as a sequence of discrete steps with typed transitions. This includes distinguishing entry points, exit points, and intermediate steps; representing unconditional and conditional transitions with associated routing logic; and linking each step to its associated task where applicable.

4.4 Reasoning Engine

The reasoning engine acts as the cognitive core of the agent, transforming high-level goals into actionable steps through multi-step reasoning and planning [LWZ+24]. In contrast to traditional models that provide immediate, single-step responses, agentic AI performs a step-by-step solution process that includes problem analysis, planning, solving, reflection and validation [Sch25].

To implement this robust cognition, systems employ structured reasoning frameworks such as Chain of Thought (CoT), Tree of Thoughts (ToT) and Graph of Thoughts

(GoT) [LWZ+24]. These architectural frameworks allow agents to explore multiple partial reasoning chains concurrently, evaluate the viability of intermediate results and perform self-correction before committing to a final action [Sch25]. Furthermore, reasoning processes can be categorized into one-step decision-making, where a task is decomposed and executed in a single logical flow and multi-step reasoning, which requires iterative reasoning cycles that maintain long-term consistency with the overall objective [GCW+24].

4.4.1 Framework Analysis

CrewAI

CrewAI provides reasoning as an explicit, configurable feature at the agent level. On a source code level, this is achieved by setting a boolean flag reasoning in the agent definition. When set to True, the agent reflects on a given task, evaluates possible approaches and refines its plan before executing an action. This reflection cycle repeats until the agent converges on a satisfactory plan or a configurable max_reasoning_attempts threshold, defined as an integer in the agent definition, is reached. Reasoning in CrewAI is therefore a framework-managed capability that is toggled per agent independently of the underlying language model [Cre25].

AutoGen

In AutoGen, reasoning is not exposed as a dedicated framework feature but is integrated into its conversation-based architectural paradigm. Whether and how an agent reasons depends primarily on the capabilities of the underlying language model. If a reasoning-capable model such as o3-mini is selected as the model client, the model itself handles multi-step reasoning internally without additional framework configuration. Alternatively, reasoning can be achieved architecturally by introducing a dedicated planning agent that decomposes a task into sub-steps using a language model before delegating them to other agents. In this case, reasoning emerges from the orchestration of multiple agents rather than from a single agent's configuration [Mic25].

LangGraph

LangGraph does not provide a built-in reasoning mechanism at the agent or node level. Since LangGraph models agentic behavior as explicit graph structures, reasoning patterns must be manually constructed by the developer through the graph topology itself. A Chain of Thought pattern, for instance, can be implemented as a sequence of nodes where each node performs one reasoning step and passes its output via the shared state to the next. More complex patterns such as reflection or self-correction require the developer to define conditional edges that loop back to previous nodes based on evaluation criteria. LangGraph therefore offers maximum flexibility in designing reasoning workflows but places the full architectural decision making on the developer, as no reasoning behavior is provided by default [Lan25].

4.4.2 Synthesis and Requirements

The framework analysis of reasoning capabilities revealed three different abstraction levels. CrewAI treats reasoning as a framework-managed, declarative feature that is toggled per agent through a boolean flag and bounded by a configurable iteration limit. AutoGen delegates reasoning either to the underlying language model's native capabilities or to the architectural composition of multiple agents, where a dedicated planning agent orchestrates task decomposition. LangGraph provides no built-in reasoning mechanism and instead requires developers to manually encode reasoning patterns through the graph topology itself.

From this analysis, the following representational requirement for reasoning can be derived:

R15 Reasoning as capability and pattern. The ontology must represent reasoning on three dimensions: (1) whether an agent has reasoning enabled as a declarative capability, (2) which reasoning pattern is employed if identifiable, and (3) whether reasoning originates from the framework, the language model's native capabilities, or the system's graph topology.

4.5 Memory Architecture

Memory provides agents with temporal continuity and the foundational knowledge required to navigate complex environments [BKN+25]. The architecture is typically divided into distinct functional durations. Short-term memory (or working memory) maintains the immediate conversational or task context within the language model's context window constraints [DBM25] [Sch25]. Long-term memory, conversely, captures persistent data across sessions, allowing the agent to recall historical interactions, user preferences and learned knowledge over time [DBM25].

Beyond duration, long-term memory can be functionally categorized into semantic memory for storing facts and reasoning paths, procedural memory for recalling specific task flows and strategies and episodic memory for retaining detailed contextual snapshots of specific past events [DBM25]. To manage and access this amount of information, agents frequently utilize Retrieval-Augmented Generation (RAG) and external vector databases. These systems retrieve relevant text snippets based on semantic similarity and append them to the agent's prompts, thereby reducing hallucinations and supporting dynamic learning [GCW+24] [Sch25]. Memory operations also involve active reflection, where agents summarize and distill past experiences to continually enhance their cognitive abilities and adaptability [LWZ+24].

4.5.1 Framework Analysis

CrewAI

CrewAI provides an integrated Memory class that supports short-term, long-term, entity and external memory types. This class uses a language model to analyze content at storage time, inferring scope, categories and importance. The memory system can be used as a standalone component, at the agent level, at the crew level and within flows.

At the crew level, all agents in a crew share the crew's memory unless an individual agent has its own. After each task, the crew automatically extracts facts from the task output and stores them. Before the next task, relevant context is recalled from memory and injected into the task prompt at runtime. At the agent level, agents can maintain private memory that is not shared with the rest of the crew. Standalone memory acts as a general-purpose knowledge base that is independent of any specific agent or crew [Cre25].

AutoGen

AutoGen provides a Memory protocol that defines a standard interface for storing and retrieving contextual information. The protocol specifies methods for adding entries, querying relevant content, updating an agent's model context and clearing the store. The simplest implementation is ListMemory, which maintains memory entries in chronological order and appends them to the model's context before each task execution.

On a source code level, memory is assigned to an agent by passing a list of memory instances during agent instantiation via the memory parameter. At runtime, the agent queries its memory stores before processing a task and the retrieved content is injected into the model context as a system message. Memory in AutoGen is therefore agent-scoped: each agent manages its own memory instances independently.

For more advanced use cases, AutoGen provides memory extensions such as ChromaDBVectorMemory and RedisMemory, which implement the same Memory protocol but use vector similarity search for retrieval rather than chronological ordering. Since all implementations conform to the same protocol, they are interchangeable at the source code level without modifying the agent's logic [Mic25].

LangGraph

LangGraph implements memory through two distinct mechanisms that map to different persistence scopes. Short-term memory is provided through *checkpointers*, which persist the graph's state across invocations within the same thread. On a source code level, a checkpointer is instantiated and passed to the graph at compile time via the checkpointer parameter. Each invocation is associated with a thread_id and the checkpointer automatically saves and restores the full graph state between turns, enabling multi-turn conversations.

Long-term memory is implemented through a separate *store* abstraction, which persists data across threads and sessions. A store is similarly passed to the graph at compile time via the store parameter. Unlike the checkpointer, which operates transparently on the entire graph state, the store must be explicitly accessed within node functions to read and write memory entries. Entries are organized into namespaces, typically scoped by user or application context [Lan25].

4.5.2 Synthesis and Requirements

CrewAI provides a unified memory system that operates at both crew and agent level, with automatic fact extraction and injection at runtime. AutoGen exposes memory through a protocol-based interface that is strictly agent-scoped, with no built-in shared memory across agents. LangGraph separates memory into two distinct mechanisms (i.e. checkpointers for short-term state persistence and stores for long-term data) both scoped to the graph as a whole and accessed explicitly within node functions.

From this analysis, the following representational requirements can be derived:

R16 Memory scope as an explicit property. The ontology must represent at which level memory is owned and accessible: agent-private, shared among a group, or global to the system.

R17 Separation of short-term and long-term persistence. The ontology must represent the lifecycle of memory, whether it persists within a single execution, across executions within a thread, or across all sessions independently of how a given framework packages its memory features.

4.6 Tool Integration

Tool integration significantly expands an agent's capabilities beyond its static, pre-trained knowledge by enabling it to interact directly with external systems and physical or digital environments [Sch25]. By dynamically invoking functionalities such as web browsers, calculators, code interpreters and specialized APIs, agents can retrieve real-time data, perform complex computations and execute software commands [LWZ+24] [Sch25]. The process of utilizing a tool involves several seamless sub-tasks: intent recognition to determine when a tool is needed, function selection to choose the most appropriate tool, parameter-value mapping to extract required arguments and the actual execution and response generation [YEL+25].

Advanced agentic systems are not just limited to using pre-existing tools; they also demonstrate the ability to autonomously select, combine and even create new tools. Because language models can generate executable code, agents can write new Python utility functions or scripts on the fly, verify them through unit tests and deploy them as custom tools to solve novel problems. This dynamic tool integration reduces computational

errors, enhances the interpretability of the agent's actions and embeds operational rules in a clear, reliable manner [Sch25].

4.6.1 Framework Analysis

CrewAI

CrewAI defines a tool as a skill or function that agents can utilise to perform actions. Tools can be assigned at two levels: at the agent level, where they become available across all tasks the agent executes and at the task level, where they override the agent's default tool set for that specific task.

On a source code level, custom tools are created either by subclassing BaseTool or by using the @tool decorator on a function. When subclassing, the developer defines a name, a description, an input schema via args_schema (a Pydantic model) and implements the _run method containing the tool's logic. When using the decorator, the function's docstring serves as the tool description that the agent uses to decide when to invoke it. CrewAI also provides a large library of pre-built tools (e.g. SerperDevTool, FileReadTool, PDFSearchTool) that can be instantiated and assigned to agents or tasks without custom implementation [Cre25].

AutoGen

AutoGen enables tool use through the AssistantAgent, which accepts tools via the tools parameter. A tool is most commonly defined as a Python function, which the agent automatically converts into a FunctionTool at runtime. The tool schema — including name, description and parameter types — is automatically generated from the function's signature and docstring and provided to the language model during execution.

Beyond individual tools, AutoGen introduces the concept of a *Workbench*, which is a collection of tools that share state and resources. The primary example is McpWorkbench, which connects to a Model Context Protocol (MCP) server and exposes its tools to the agent. A workbench is passed to the agent via the workbench parameter and operates alongside any individually assigned tools [Mic25].

LangGraph

LangGraph implements tool use through LangChain's tool infrastructure. Tools are defined as Python functions using the @tool decorator, where the function's docstring serves as the description and type hints define the input schema. For more complex inputs, a Pydantic model or JSON schema can be specified via the args_schema parameter.

Tools are integrated into the graph through a prebuilt ToolNode, which handles tool execution, parallel tool calls and error handling. The ToolNode is added to the graph like any other node and connected to the model node via a conditional edge (tools_condition) that routes execution to the tool node only when the model generates a tool call [Lan25].

4.6.2 Synthesis and Requirements

All three frameworks share the fundamental concept of a tool as a callable function with a name, description and typed input schema that is exposed to a language model. The definition mechanism is also convergent: all frameworks support defining tools as decorated Python functions with auto-generated schemas. The key differences lie in how tools are assigned, scoped and integrated into the execution flow.

From this analysis, the following representational requirements can be derived:

R18 Tool as a framework-independent concept. The ontology must represent tools as entities with a name, description and input schema, independent of the framework-specific definition mechanism (decorator, subclass, or function).

R19 Tool assignment scope. The ontology must capture at which level a tool is bound: to an individual agent (all frameworks), to a specific task (CrewAI), or to a graph node (LangGraph).

4.7 Guardrails and Governance Mechanisms

Because agentic AI operates with high autonomy in dynamic environments, robust guardrails are essential to ensure that systems act safely, predictably and in alignment with human values. Guardrails consist of embedded technical constraints. Including goal safety protocols, schema validators and fail-safe mechanisms—that monitor and validate agent outputs before execution, preventing harmful actions or deviations from assigned objectives. By incorporating node validation, retry logic and early-stage trust layers, developers can secure agent workflows and mitigate risks like malicious code execution or data privacy breaches [AKA25] [DBM25].

On a broader systemic level, governance frameworks dictate the degree of human oversight and accountability required for agentic operations. This is frequently implemented through Human-in-the-Loop (HITL) architectures, where a human must explicitly approve critical decisions, or Human-on-the-Loop (HOTL) designs, where humans monitor autonomous execution with the power to intervene via override protocols. Governance also heavily relies on Explainable AI (XAI) techniques and comprehensive audit trails to log reasoning steps and tool invocations, ensuring that the agent's decision-making process remains transparent, legally compliant and accountable when errors occur [AKA25].

4.7.1 Framework Analysis

CrewAI

CrewAI implements guardrails at the task level through the guardrail and guardrails parameters. Two types are supported: function-based guardrails, which are Python functions that receive a TaskOutput and return a tuple indicating success or failure with

feedback and LLM-based guardrails, which are string descriptions that the agent's language model evaluates against the output. Multiple guardrails can be chained sequentially, with each receiving the output of the previous one. A configurable guardrail_max_retries parameter limits how many times an agent reattempts a task after guardrail failure [Cre25].

AutoGen

AutoGen does not provide a dedicated guardrail mechanism at the task or agent level. Output validation must be implemented by the developer, typically through a dedicated critic agent that evaluates outputs and provides feedback within the team's conversational flow. This is a common architectural pattern in AutoGen rather than a framework feature: a validator agent is included as a team participant and a TextMentionTermination condition (e.g. listening for "APPROVE") controls when execution stops.

Human-in-the-loop is supported through two complementary mechanisms. First, a UserProxyAgent can be included as a participant in a team, prompting the user for input during its turn via a configurable input function. This approach is synchronous and blocks the team's execution until the user responds. Second, the team can be configured to pause after a set number of agent turns using the max_turns parameter, or to stop when an agent issues a HandoffMessage targeting the user, detected by a HandoffTermination condition. In both cases, the team preserves its state after stopping and can be resumed with user feedback passed as the task parameter in the next call to run [Mic25].

LangGraph

LangGraph does not provide built-in guardrails but offers comprehensive mechanisms for human-in-the-loop through its interrupt function. Unlike static breakpoints (which pause before or after specific nodes at compile time), interrupts are dynamic: they can be placed anywhere within a node's logic and can be conditional based on application state. When interrupt is called, graph execution is suspended, the current state is saved via the checkpointer and a payload is surfaced to the caller. Execution resumes when the caller re-invokes the graph with a Command(resume=...) containing the human's response, which becomes the return value of the interrupt call inside the node.

This mechanism supports several human-in-the-loop patterns at the source code level: approval workflows where a node pauses for a binary approve/reject decision, review-and-edit patterns where a human can modify LLM outputs before they are committed to state and input validation loops where an interrupt repeatedly prompts the user until valid input is received. Interrupts can also be placed inside tool functions, causing the tool to pause for approval whenever it is invoked.

Guardrail-like validation must be implemented as a dedicated node in the graph that inspects the state and routes execution to either the next step or back to the producing node via conditional edges [Lan25].

4.7.2 Synthesis and Requirements

The frameworks diverge significantly in how they handle validation and human oversight. CrewAI provides guardrails as a first-class, configurable feature at the task level with built-in retry logic and supports human-in-the-loop through task-level flags and flow-level decorators. AutoGen has no dedicated guardrail mechanism; validation is achieved through critic agents participating in the conversational flow, while human-in-the-loop is supported through UserProxyAgent participation, max_turns pausing and HandoffTermination conditions. LangGraph requires guardrail logic to be encoded as validation nodes with conditional edges and provides a dynamic interrupt function that can be placed anywhere within node logic for human review, approval, or input validation.

From this analysis, the following representational requirements can be derived:

R20 Guardrails as a task-level property. The ontology must represent that a task or step has associated validation logic, regardless of whether it is implemented as a task parameter (CrewAI), a validator agent (AutoGen), or a conditional node (LangGraph). The representation captures that validation exists and what criteria it checks, without modeling the implementation mechanism itself.

R21 Human-in-the-loop as an explicit checkpoint. The ontology must represent points in the workflow where human review or input is required. This includes CrewAI's human_input flag and @human_feedback decorator, AutoGen's UserProxyAgent and HandoffTermination targeting the user and LangGraph's interrupt calls within nodes. The semantic representation captures the presence and position of human checkpoints so that the transformation method can reconstruct an equivalent mechanism in the target framework.

4.8 Consolidated Requirements

The preceding sections analyzed how each agentic AI concept is realized across CrewAI, AutoGen, and LangGraph at the source code level. From the synthesis of each concept area, a total of 21 representational requirements were derived. Each requirement specifies what the semantic representation must capture to enable framework-independent modeling of the corresponding concept. Table 4.2 consolidates these requirements across all seven concept areas. Together, they define the scope and acceptance criteria for the ontology engineering phase that follows.

Table 4.2: Representational Requirements for a Semantic Model of Agentic AI Systems

ID	Concept	Requirement
R1	Agent Def.	**Framework-independent agent identity.** The ontology must represent agent identity as a separable concept that can be populated either from explicit configuration attributes (as in CrewAI and AutoGen) or inferred from prompt-level content within computational nodes (as in LangGraph).
R2	Agent Def.	**Distinction between orchestration-relevant and model-relevant attributes.** The ontology must distinguish between agent attributes that serve the orchestration layer and those that instruct the language model. A single agent may carry both types of directive simultaneously.
R3	Agent Def.	**Prompt decomposition into structured components.** The ontology must decompose natural language prompt content into structured components so that equivalent prompts can be reconstructed for different target frameworks from these components.
R4	Agent Def.	**Flexible attribute cardinality.** The ontology must define a minimal set of identity properties while accommodating framework-specific extensions without loss of information.
R5	Agent Def.	**Agent functional typing.** The ontology must represent the functional role an agent plays within the coordination structure by distinguishing between general-purpose agents, manager/orchestrator agents and agents that serve as proxies for human input.
R6	Perception/Task Def.	**Task as a first-class ontological concept.** The ontology must represent tasks as distinct entities with their own properties, even when the source framework does not model them explicitly.
R7	Perception/Task Def.	**Agent-Task assignment.** The ontology must represent the relationship between a task and the agent responsible for its execution, including explicit assignment, implicit assignment through orchestration and structural assignment through graph topology.
R8	Perception/Task Def.	**Data dependencies.** The ontology must capture ordering and data dependencies between tasks and normalize these into a unified dependency model.
R9	Perception/Task Def.	**Expected outputs.** The ontology must represent the expected output of a task, both as a natural language description and, where available, as a structured schema definition.
R10	Coordination	**Coordination pattern as a named concept.** The ontology must represent coordination patterns (sequential, hierarchical, round-robin, swarm, selector-based) as identifiable concepts regardless of whether they are selected from a preset or manually constructed in a graph.

Continued on next page

ID	Concept	Requirement
R11	Coordination	**Agent-to-coordination binding.** The ontology must capture which agents participate in a coordination structure and how they are bound to it.
R12	Coordination	**Termination conditions.** The ontology must capture termination semantics independently of framework-specific mechanisms.
R13	Coordination	**Multi-level coordination.** The ontology must represent nested coordination structures so that a CrewAI flow containing multiple crews can be captured without flattening the hierarchy, while also representing single-level coordination.
R14	Coordination	**Workflow step structure with typed transitions.** The ontology must represent the execution structure as a sequence of discrete steps with typed transitions, distinguishing entry points, exit points, intermediate steps, unconditional and conditional transitions with associated routing logic.
R15	Reasoning	**Reasoning as capability, pattern and origin.** The ontology must represent whether an agent has reasoning enabled, which reasoning pattern is employed if identifiable and whether reasoning originates from the framework, the model or the system topology.
R16	Memory	**Memory scope as an explicit property.** The ontology must represent at which level memory is owned and accessible: agent-private, shared among a group or global to the system.
R17	Memory	**Separation of short-term and long-term persistence.** The ontology must represent the lifecycle of memory independently of how a given framework packages its memory features.
R18	Tools	**Tool as a framework-independent concept.** The ontology must represent tools as entities with a name, description and input schema, independent of the framework-specific definition mechanism.
R19	Tools	**Tool assignment scope.** The ontology must capture at which level a tool is bound: to an individual agent, to a specific task or to a graph node.
R20	Guardrails	**Guardrails as a task-level property.** The ontology must represent that a task has associated validation logic, regardless of whether it is implemented as a task parameter, a validator agent or a conditional node.
R21	Guardrails	**Human-in-the-loop as an explicit checkpoint.** The ontology must represent points in the workflow where human review or input is required, capturing the presence and position of human checkpoints.

CHAPTER 5

A Framework-Agnostic Semantic Representation

The previous chapter analyzed how agentic AI concepts are realized across three frameworks and derived a set of representational requirements from those findings. These requirements describe what a framework-independent semantic representation must be able to express in order to support interoperability. This chapter presents the ontology developed to meet those requirements.The ontology is the first of the two artefacts of this thesis. Its purpose is specific: it serves as the common intermediate representation into which framework-specific source code is extracted and from which framework-specific source code is generated.

The ontology is developed following the Ontology Development 101 methodology proposed by [NM01]. The first step of this methodology is to determine the domain and scope of the ontology. The domain is agentic AI systems and the ability to represent cross-framework implementation details. The scope is defined through competency questions, as introduced by [GF95].

5.1 Competency Questions

Competency questions are natural language queries that a fully populated ontology must be able to answer. They act as boundary conditions: if a populated ontology instance cannot answer a competency question, either the ontology is missing a required concept or the extraction has not captured it [NM01]. In this thesis, the competency questions are derived directly from the requirements established in Chapter 4. The corresponding competency question can be found in 5.1.

Table 5.1: Competency Questions derived from Representational Requirements

CQ	Req.	Concept	Competency Question
CQ-1	R1	Agent Def.	Which agents are defined in this system and what identifies each one?
CQ-2	R2	Agent Def.	Which of an agent's prompts guide the orchestration layer and which instruct the language model?
CQ-3	R3	Agent Def.	What are the instruction, context and output indicator components of a given agent's prompt?
CQ-4	R4	Agent Def.	Which agent properties are explicitly specified and which are left undefined across different framework representations?
CQ-5	R5	Agent Def.	Is a given agent a general-purpose worker, a manager or a proxy for human input?
CQ-6	R6	Perception	What tasks does this system define and what does each task instruct the assigned agent to do?
CQ-7	R7	Perception	Which agent is responsible for a given task and how was that assignment determined?
CQ-8	R8	Perception	Which tasks must complete before a given task can start and what is the nature of that dependency?
CQ-9	R9	Perception	What output is a task expected to produce and is a structured output schema defined for it?
CQ-10	R10	Coordination	What coordination pattern does a given team follow?
CQ-11	R11	Coordination	Which agents belong to a given team and how are they bound to its coordination structure?
CQ-12	R12	Coordination	Under what conditions does a team's execution stop?
CQ-13	R13	Coordination	Does a team contain sub-teams and if so how are they nested?
CQ-14	R14	Coordination	What is the step-by-step execution structure of a team's workflow and where does it branch conditionally?
CQ-15	R15	Reasoning	Does an agent use reasoning, what pattern does it follow and where does that capability originate?
CQ-16	R16	Memory	Is an agent's memory private, shared with the team or accessible system-wide?
CQ-17	R17	Memory	Does a given memory persist only during execution, across threads or across all sessions?
CQ-18	R18	Tools	What tools are available in this system and what inputs does each one expect?
CQ-19	R19	Tools	Is a given tool bound at the agent level or overridden for a specific task?

Continued on next page

CQ	Req.	Concept	Competency Question
CQ-20	R20	Guardrails	Does a task have validation guardrails and what type of check do they perform?
CQ-21	R21	Guardrails	At which points in the workflow must a human provide input or approval?

5.2 Scoping Decision

It is also important to be clear about what the ontology does not capture. Runtime state, execution history and dynamic behavior that only materialities when a system runs are outside scope. This includes conversation histories, dynamically selected speakers in AutoGen's SelectorGroupChat and the actual values passed through LangGraph's shared state.

5.3 Reuse Evaluation

The ontology is not built from scratch. AgentO [EKEK26] provides the conceptual foundation, defining core classes such as `LLMAgent`, `Task`, `Tool`, `Prompt` and `WorkflowStep` along with 22 object properties and 10 datatype properties. Where AgentO's existing vocabulary satisfies a requirement, it is reused without modification. Where it falls short, new classes and properties are introduced. All extensions are strictly additive: no existing AgentO class is removed or structurally modified and no existing property has its domain or range altered.

To determine where extensions are needed, we evaluated each requirement from Chapter 4 against AgentO's vocabulary. Table 5.2 summarises the results. Of the 21 requirements, AgentO fully covers 4 (R1, R4, R6, R11) through its existing agent identity, flexible cardinality, task and team membership classes. Eight requirements are partially covered: the relevant classes exist but lack properties needed for cross-framework transformation, such as distinguishing orchestration-facing from model-facing prompts (R2) or recording how task-agent binding was determined (R7). The remaining nine requirements address concepts entirely absent from AgentO, including coordination patterns, termination conditions, reasoning strategies, guardrails and human-in-the-loop checkpoints. The extensions derived from this analysis are detailed in the following section.

Table 5.2: AgentO competency coverage against derived requirements.

Requirement	**Cov.**	**Key extension**
Agent Definition		
R1 Agent identity	•	—
R2 Directive distinction	○	`hasDirectiveFunction`
R3 Prompt decomposition	○	`hasSourceAttribute`
R4 Flexible cardinality	•	—
R5 Agent functional typing	○	`agentType`
Perception Layer		
R6 Task as first-class	•	—
R7 Task-agent assignment	○	`hasDelegationStrategy`
R8 Task dependencies	×	`dependsOn`
R9 Task output	○	`hasExpectedOutput`, `hasOutputSchema`
Coordination		
R10 Coordination patterns	×	`CoordinationPattern` hierarchy
R11 Agent-coordination binding	•	—
R12 Termination conditions	×	`TerminationCondition` hierarchy
R13 Multi-level coordination	×	`Orchestration`, `AgenticSystem`
R14 Workflow step structure	○	`ConditionalStep`, `hasRoutingLogic`
Reasoning		
R15 Reasoning capability	×	`ReasoningPattern` hierarchy
Memory		
R16 Memory scope	○	`MemoryBinding`, `hasMemoryScope`
R17 Persistence separation	○	`hasPersistenceScope`
Tools		
R18 Tool as concept	○	`hasInputSchema`
R19 Tool assignment scope	×	`taskToolUsage`
Guardrails & Governance		
R20 Guardrails	×	`Guardrail`, `hasGuardrail`
R21 Human-in-the-loop	×	`HumanCheckpoint`

• = fully covered (4) ○ = partially covered (8) × = not covered (9)

5.4 Defining Classes and Properties

5.4.1 Agent Definition

5.4.2 Perception

5.4.3 Coordination

5.4.4 Reasoning

5.4.5 Memory

5.4.6 Tools

5.4.7 Guardrails and Goverance

CHAPTER 6

Bidirectional Transformation Method

This chapter presents the transformation method that links framework-specific source code to the semantic representation developed in Chapter **??**. The method is grounded in the OSCIN framework [dAZS21] for ontology-based source code interoperability, with two extensions motivated by the specific characteristics of the agentic AI domain. Section 6.1 maps the OSCIN phases to this thesis context and states the scoping decisions. Section **??** presents the extraction direction: three framework-specific extractors that transform Python source code into ontology instances. Section **??** presents the generation direction: a template-based code synthesiser that transforms ontology instances into executable CrewAI code. Section **??** provides a systematic characterisation of information loss across all transformation paths.

6.1 Method Overview

6.1.1 OSCIN Instantiation

The OSCIN method [dAZS21] defines five phases for achieving ontology-based source code interoperability: (1) Specification, which defines the transformation scope; (2) Subdomain Semantic Abstraction, which constructs the domain ontology; (3) Language Syntactic Abstraction, which defines parsers for each source language; (4) Language Semantic Abstraction, which maps syntactic structures to ontology instances; and (5) Application Perspective, which implements domain-specific operations over the populated ontology. Table 6.1 maps these phases to the thesis.

Two extensions distinguish this instantiation from the original OSCIN method. First, OSCIN assumes that all semantic content is recoverable from syntax through deterministic mapping tables. In the agentic AI domain, natural language content is embedded in

Table 6.1: Mapping of OSCIN phases to the thesis context.

OSCIN Phase	Thesis Chapter	Instantiation
1. Specification	Ch. 3 (Methodology)	Subdomain: agentic AI systems. Language: Python. Frameworks: CrewAI, LangGraph, AutoGen v0.4. Perspective: cross-framework interoperability.
2. Subdomain Semantic Abstraction	Ch. **??**	AgentO-Ext ontology: 5 new classes, 11 object properties, 26 data properties extending AgentO.
3. Language Syntactic Abstraction	§**??**	Python ast module provides the parser. Framework-specific AST patterns are defined per extractor.
4. Language Semantic Abstraction	§**??**	Hybrid: deterministic AST mapping for structural constructs + LLM interpretation for natural language content. **Extension of OSCIN.**
5. Application Perspective	§**??**	Code generation from ontology instances into CrewAI. **Extension of OSCIN** (generation direction).

source code as unstructured strings (system prompts, agent descriptions, task instructions) whose semantic decomposition cannot be achieved through syntactic analysis alone. The extraction method therefore introduces a second semantic abstraction layer: constrained LLM interpretation applied only to content that AST cannot structurally decompose. This hybrid approach preserves OSCIN's determinism for structural constructs while extending it to handle the natural language content that is characteristic of LLM-based systems.

Second, OSCIN's Application Perspective is designed for read-only analysis (e.g., code smell detection via SPARQL queries). This thesis extends it toward code generation: ontology instances are transformed into executable source code for a target framework. This extends OSCIN from a one-directional analysis tool toward round-trip engineering.

6.1.2 Scoping Decisions

CHAPTER 7

Evaluation

CHAPTER 8

Discussion

CHAPTER 9

Conclusion

Overview of Generative AI Tools Used

List of Figures

List of Tables

List of Algorithms

Bibliography

[AKA25] Deepak Acharya, Karthigeyan Kuppan, and Divya B Ashwin. Agentic ai: Autonomous intelligence for complex goals – a comprehensive survey. *IEEE Access*, PP:1–1, 01 2025.

[BKN+25] Ajay Bandi, Bhavani Kongari, Roshini Naguru, Sahitya Pasnoor, and Sri Vidya Vilipala. The rise of agentic ai: A review of definitions, frameworks, architectures, applications, evaluation metrics, and challenges. *Future Internet*, 17(9), 2025.

[Bot25] V. Botti. Agentic ai and multiagentic: Are we reinventing the wheel?, 2025.

[CLH+25] Shuaihang Chen, Yuanxing Liu, Wei Han, Weinan Zhang, and Ting Liu. A survey on llm-based multi-agent system: Recent advances and new frontiers in application, 2025.

[Cre25] CrewAI Inc. CrewAI documentation. `https://docs.crewai.com`, 2025. Accessed: April 2026.

[dAZS21] Camila Zacché de Aguiar, Félix Zanetti, and Vitor E. Silva Souza. Source code interoperability based on ontology. In *Proceedings of the XVII Brazilian Symposium on Information Systems*, SBSI '21, New York, NY, USA, 2021. Association for Computing Machinery.

[DBM25] Hana Derouiche, Zaki Brahmi, and Haithem Mazeni. Agentic ai frameworks: Architectures, protocols, and design challenges, 2025.

[EKEK26] Andreas Ekelhart, Kabul Kurniawan, Fajar J. Ekaputra, and Elmar Kiesling. Agento: An ontology for modeling agentic ai systems. In *The Semantic Web: 23th International Conference, ESWC 2026 (accepted for publication)*, 2026.

[GCW+24] Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, Shichao Pei, Nitesh V. Chawla, Olaf Wiest, and Xiangliang Zhang. Large language model based multi-agents: A survey of progress and challenges, 2024.

[GF95] Michael Grüninger and Mark Fox. Methodology for the design and evaluation of ontologies. 07 1995.

[HZC+24] Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. Metagpt: Meta programming for a multi-agent collaborative framework, 2024.

[Lan25] LangChain Inc. LangGraph documentation. `https://langchain-ai.github.io/langgraph/`, 2025. Accessed: April 2026.

[LWZ+24] Xinyi Li, Sai Wang, Siqi Zeng, Yu Wu, and Yi Yang. A survey on llm-based multi-agent systems: workflow, infrastructure, and challenges. *Vicinagearth*, 1, 10 2024.

[Mic25] Microsoft Research. AutoGen documentation. `https://microsoft.github.io/autogen/`, 2025. Accessed: April 2026.

[NM01] Natalya F. Noy and Deborah L. McGuinness. Ontology development 101: A guide to creating your first ontology. *Knowledge Systems Laboratory*, 32, January 2001.

[NSSP26] Ume Nisa, Muhammad Shirazi, Mohamed Ali Saip, and Muhammad Syafiq Mohd Pozi. Agentic ai: The age of reasoning—a review. *Journal of Automation and Intelligence*, 5(1):69–89, 2026.

[Sch25] Johannes Schneider. Generative to agentic ai: Survey, conceptualization, and challenges, 2025.

[WJ95] Michael Wooldridge and Nicholas R. Jennings. Intelligent agents: theory and practice. *The Knowledge Engineering Review*, 10(2):115–152, 6 1995.

[Woo09] Michael Wooldridge. *An Introduction to MultiAgent Systems*. John Wiley Sons, 6 2009.

[WZ25] Christopher Wissuchek and Patrick Zschech. Exploring agentic artificial intelligence systems: Towards a typological framework, 2025.

[YEL+25] Asaf Yehudai, Lilach Eden, Alan Li, Guy Uziel, Yilun Zhao, Roy Bar-Haim, Arman Cohan, and Michal Shmueli-Scheuer. Survey on evaluation of llm-based agents, 2025.